Keypoint-Guided Efficient Pose Estimation and Domain Adaptation for Micro Aerial Vehicles

Ye Zheng, Canlun Zheng, Jiahao Shen, Peidong Liu, Shiyu Zhao

***Abstract*—Visual detection of micro aerial vehicles (MAVs) is an important problem in many tasks such as vision-based swarming of MAVs. This paper studies vision-based 6D pose estimation to detect a 3D bounding box of a target MAV and then estimate its 3D position and 3D attitude. The 3D attitude information is critical to better estimate the target's velocity since the attitude and motion are dynamically coupled. In this paper, we propose a novel 6D pose estimation method, whose novelties are threefold. First, we propose a novel centroid point-guided keypoint localization network that outperforms the state-of-the-art methods in terms of both accuracy and efficiency. Second, while there are no publicly available real-world datasets for 6D pose estimation for MAVs up to now, we propose a high-quality dataset based on an automatic dataset collection method. Third, since the dataset is collected in an indoor environment but detection tasks are usually in outdoor environments, we propose a self-training-based unsupervised domain adaption method to transfer the method from indoor to outdoor. Finally, we show that the estimated 6D pose especially the 3D attitude can significantly help improve the target's velocity estimation.**

***Index Terms*—6D pose estimation, micro aerial vehicles, unsupervised domain adaptation**

I. Introduction

Visual detection of micro aerial vehicles (MAVs) [1]–[3] is an important problem in various tasks such as detection of malicious MAVs [4], [5] and see-and-avoid systems for MAVs [6]. This paper is particularly motivated by the task of *vision-based swarming of MAVs*. Although MAV swarming has been studied extensively so far, most studies focus on the *control aspect* to design cooperative controllers based on the assumption that each MAV can obtain the information of their neighbors via certain ways such as wireless communication [7]–[9]. By contrast, much fewer studies consider the *sensing aspect* so that each MAV uses onboard visual sensors to detect their neighbors [10]–[12]. Visual detection of MAVs has not been well studied up to now and remains a critical technical challenge for realizing high-performance MAV swarming in practice.

Ye Zheng is with the School of Engineering at Westlake University and the College of Computer Science and Technology at Zhejiang University, Hangzhou, China. (Email: zhengye@westlake.edu.cn)

Canlun Zheng, Jiahao Shen and Peidong Liu are with the School of Engineering at Westlake University, Hangzhou, China. (Email: zhengcanlun@westlake.edu.cn; shenjiahao@westlake.edu.cn; liupeidong@westlake.edu.cn)

Shiyu Zhao is with the Research Center for Industries of the Future and the School of Engineering at Westlake University, Hangzhou, China. (Email: zhaoshiyu@westlake.edu.cn)

This work was supported by the STI 2030-Major Projects (Grant No. 2022ZD0208800) and the Research Center for Industries of the Future at Westlake University (Grant No. WU2022C027). (Corresponding author: Shiyu Zhao)



Fig. 1. An example of air-to-air MAV 6D pose estimation with a monocular camera. Our method detects a 3D bounding box surrounding the target MAV and then estimates its 6D pose including position and attitude.

Unlike many existing methods that detect 2D bounding boxes of target MAVs [2], [5], our present work considers 6D pose estimation. That is to detect the 3D bounding box of the target MAV and then estimate its 6D pose including the 3D position and 3D attitude (see Fig. 1). The reason that we are particularly interested in the *3D attitude* information of the target MAV is that this information is important to better estimate the target's motion. In particular, the acceleration of a MAV is tightly coupled with its attitude. If we know the attitude of the target MAV, then we can better estimate its motion by considering the nonlinear flight dynamics [13]. Attitude estimation is also important because many high-performance cooperative control methods relying on accurate attitude information [7], [14], [15], especially those for attitude synchronization [16], [17].

6D pose estimation of general objects has attracted increasing research attention in recent years (a detailed review is given in Section II-A). Existing methods can be classified into *instance-level* and *category-level*. The difference between the two types of methods is that the instance-level focuses more on target objects whose information is known in advance [18], [19], whereas the category-level focuses more on detecting unseen target objects based on large-scale datasets [20], [21]. Since we consider cooperative swarming where the MAVs are known in advance, we focus on the instance-level case so that the prior information of the MAVs can be fully exploited.

Although there are many methods of instance-level pose estimation of general objects, it is difficult to apply them to the case of MAVs due to the following challenges.

The first challenge is about real-world datasets. On the one hand, it is necessary to have a real-world dataset that contains

labeled images of MAVs together with their associated high-quality pose information in order to train an effective 6D pose estimation method. However, no real-world dataset is publicly available to the best of our knowledge although there exist studies on 6D pose estimation of MAVs [22]–[24]. On the other hand, it is nontrivial to collect a real-world dataset, especially in outdoor environments due to the relatively low-accuracy sensors such as the GPS and barometer. Although indoor motion capture systems can provide high-accuracy groundtruth, how to transfer the trained method from indoor to outdoor is a problem that must be studied.

The second challenge is about high efficiency. On the one hand, the vision-based method must be executed in the onboard computers of MAVs that usually have limited computational capabilities. On the other hand, since the target MAV may maneuver fast, it is necessary to ensure high-frequency detection and estimation. Due to the two aspects, many state-of-the-art (SOTA) methods are hardly applicable to MAV tasks as verified by our experiments.

The contributions of this paper are summarized as follows.

1) A new method called MAV6D is proposed to efficiently estimate the 6D pose of a given MAV. The idea of MAV6D is to first localize the pre-defined keypoints of a given MAV in an image and then estimate its 6D pose by the Perspective-n-Point (PnP) technique [25]. We propose a novel centroid point-guided keypoint localization network (CPKL-Net) that can directly recognize and localize the keypoints rather than require an additional object detection network to detect the MAV first [24]. CPKL-Net can significantly decrease the detection latency and we show by experiments that its efficiency is 16 times faster than the SOTA method ROPE [26]. Moreover, we also propose a 2D and 3D combination loss for learning keypoint regression. With this loss, CPKL-Net achieves SOTA localization accuracy. Experimental results demonstrate that CPKL-Net outperforms the SOTA methods ROPE [26], YOLO6D [27], and EfficientPose [28] in terms of both accuracy and efficiency.

2) To train the proposed network, we propose a high-quality dataset in an indoor environment based on an automatic dataset collection method. The hardware platform of this method consists of a motion capture system VICON, a monocular camera, and a MAV. Each of the hardware defines a coordinate frame. Accurate calibration of the transformations between the three coordinate frames is critical to generate accurate pose labels. To this end, we introduce a reprojection error minimization method. Thanks to the automatic method, more than 57,000 high-quality labeled images can be generated efficiently. The dataset contains a variety of flight conditions such as different attitude values and various viewing angles.

3) Since the dataset is collected in an indoor environment but detection tasks are usually in outdoor environments, we propose a self-training-based unsupervised domain adaption (UDA) method to transfer the model from indoor to outdoor. In particular, we generate pseudo-labels by training the CPKL-Net with a mixed domain dataset and conduct network re-training as well as pseudo-label screening to obtain high-quality pseudo-labels and a better performance network. This method effectively addresses the problem of MAV 6D pose estimation when outdoor datasets are unavailable. It only needs a small number of background images and unlabeled data of the target domain to realize the domain transfer. Experimental results verify the effectiveness of the proposed method.

4) When employing the traditional Kalman filter method (KF) for agile MAV state estimation, a considerable lag occurs. In MAV swarm tasks such as MAV formation and collaborative operations, where MAVs exhibit high-speed flight, this lag may cause safety problems. To this end, we introduce a pose Kalman filter method (Pose-KF), which incorporates MAV attitude information obtained from the MAV6D method into the state-transition model. Extensive real-world experiments have verified the effectiveness of the Pose-KF. By contrast to the KF, this method significantly improves the accuracy of the state-transition model and the response speed of the filter in estimating higher-order information. This is beneficial for tracking highly maneuverable targets in the 3D space.

II. Related Work

This section gives a review of 6D pose estimation methods and UDA methods. Here, only the methods based on monocular vision are reviewed.

A. 6D Pose Estimation

6D Pose Estimation of General Objects: The existing 6D pose estimation methods can be classified into instance-level and category-level [21]. The instance-level focuses more on target objects whose information is known in advance [18], [19], whereas the category-level focuses more on detecting unseen target objects based on large-scale datasets [20], [21]. In the case of vision-based swarming, the MAVs are cooperative and known in advance. Thus, we focus on the instance-level pose estimation method. This method can be further classified into two types.

The first type is template-based. These methods are based on the computer-aided design (CAD) model of an object. For example, the works in [18] and [29] utilizes the CAD model and the renderer to generate a database, which contains multi-view images of the object and the groundtruth poses. The 6D pose of an object is obtained by matching the object feature in the query image with the object features in the database. The template-based is sensitive to occlusion in general and has relatively low efficiency due to an additional object detection method.

The second type is keypoint-based. This type of method employs keypoints to construct the 2D-3D correspondence and obtain the 6D pose by using PnP algorithms. For example, the work in [19] proposes DAKDN to localize keypoints. It defines keypoints on the object. Some other works define the keypoints outside the object. For example, the works in [30], [27], and [31] couples classification and regression branches to localize the keypoints. In addition, due to the errors in keypoint projection, the network may diverge in training. To address this problem, the work in [32] adopts the real distance of 3D space [33] to supervise the learning process.

6D Pose Estimation of MAVs: There are few works in the field of vision-based MAV 6D pose estimation. The work in

[22] adopts the keypoint-based method. It presents a relational graph network to detect the keypoints. However, due to the noises caused by keypoint detection, an additional Kalman filter is required to refine the estimated poses. The work in [24] utilizes the 6D pose estimation method proposed in [34]. The running speed of this method is about 3 FPS, which is difficult to implement on MAVs. In addition, the work in [23] introduces a MAV pose dataset as well as a template-based method to complete MAV 6D pose estimate task. However, this is a simulation dataset. There is no real-world dataset publicly available now.

B. Unsupervised Domain Adaptation

UDA is used to transfer knowledge of a model learned from a source domain with rich supervision to an unseen and unlabeled target domain. In the previous works, the main paradigm is domain alignment, which aims to minimize the discrepancy between the source and target feature distributions. Although these methods have been proven to be effective to a certain extent, they have limitations in learning the invariant representations of domain shift [35], [36]. Another method is the self-training-based method [37], [38], which is proposed as a promising alternative to domain alignment. It utilizes the potentially meaningful training signal from the target domain by iteratively adding the target domain samples into the training dataset.

Recently, the self-training-based method has dominated the UDA tasks [39]. For example, the works in [40] and [36] improves the classification performance by denoise the pseudo-labels. In addition, the self-training-based method has also been applied to other computer vision tasks. The works in [41]–[43] proposes a self-training pipeline for UDA on 3D object detection and semantic segmentation. In this paper, we propose a self-training-based UDA method to transfer the MAV 6D pose estimation method trained on the indoor dataset to outdoor environments.

III. MAV6D and Dataset Collection

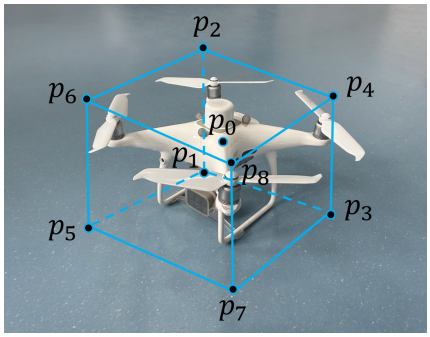
In this section, we propose a new method, named MAV6D, for estimating the 6D pose of a given quadcopter MAV. We also present a new dataset along with an automatic dataset collection method.

A. The MAV6D Method

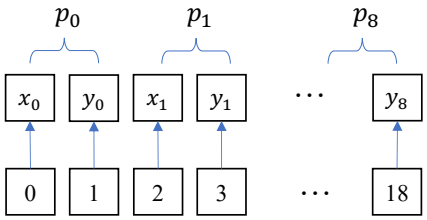
The MAV6D method consists of two steps. In the first step, a new CPKL-Net is proposed to localize nine keypoints of a given MAV in the image. In the second step, the keypoints are used to estimate the 6D pose of the MAV based on the PnP method. In contrast to other methods, this method does not require an additional object detection method to localize the MAV in the image, which saves the computational cost.

Keypoint Definition: Nine keypoints are defined for any given quadcopter MAV. The keypoints consist of one centroid point and eight corner keypoints (see Fig. 2(a)).

To localize the keypoints in the image, our idea is to utilize the centroid point to help search the other eight corner keypoints. This idea offers two advantages. Firstly, the centroid point corresponds to the distinctive appearance features of the MAV, making it a reliable feature point for MAV localization in the image. While the other eight corner points may be subject to occlusion by the MAV's body, the center point remains visible as it cannot be self-occluded. Secondly, since the MAV is a rigid body, the relative geometry of the corner keypoints remains invariant with respect to the centroid point. This enables us to localize the corner keypoints in the vicinity of the centroid point after its localization.



(a) Keypoints definition.



(b) Keypoints regression in the image.

Fig. 2. Keypoint definition and keypoint regression in the image. (a) There are 9 keypoints of a MAV, including 1 centroid point (numbered p_0) and 8 corner points (numbered $p_1 \sim p_8$). (b) The $(0, 1, 2, 3, ..., 18)$ is a grid cell vector of keypoint regression. Each grid cell element is responsible for predicting the coordinate of a fixed keypoint. For example, element 0 only predict the x_0, which is the coordinate of p_0 in the X direction.

Network Structure: The architecture of CPKL-Net is illustrated in Fig. 3. To speed up the inference process, we adopt convolution operation with the convolution kernel size of 1×1. The EfficientRep backbone and Rep-PAN [44] are used to extract and fuse features of the input image, respectively. The EfficientRep is an efficient re-parameterizable backbone built based on the RepBlock structure of RepVGG [45]. The Rep-PAN is the modified PAN topology [46] from YOLOv4 [47]. We concatenate the three output feature maps of Rep-PAN, which are with different scales, after downsampling and upsampling operations. This process is termed multi-scale feature map fusion (MFMF). Through processing the concatenated feature map with the convolution operations, the keypoints in the image can be obtained. Specifically, the RGB image with the size of 640×480 pixels is taken as input and it is divided into 40×30 grid cells. The centroid point in each grid cell of the input image will be predicted by the corresponding grid cell of the output 3D tensor, including its class, confidence, and position. The other corner keypoints are bundled together with the centroid point as a group of points.

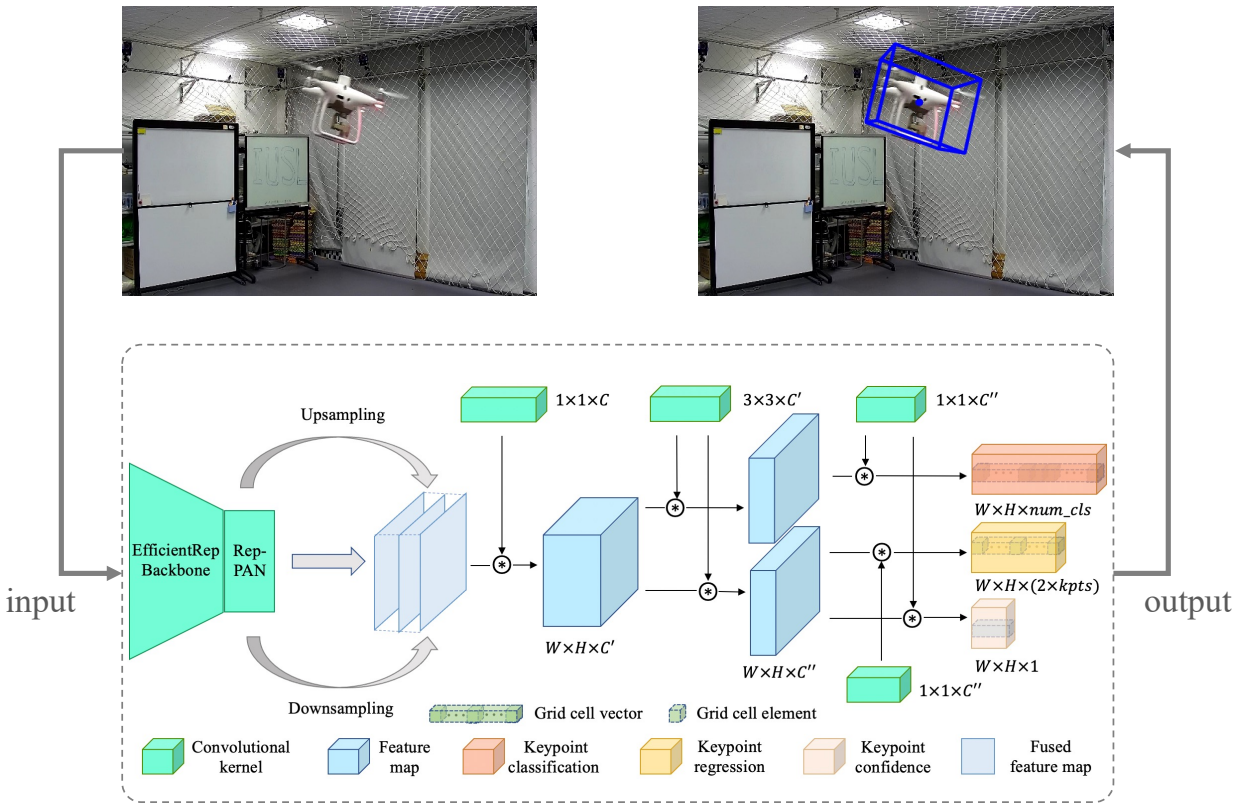


Fig. 3. Overview of the proposed keypoint localization network. Given an image, the proposed network predicts the predefined keypoints in the image. The ⊛ denotes the convolution operation. The number of convolutional kernels varies in different layers. This is represented by different sizes of convolutional kernels in this figure. The num_cls and $kpts$ are defined as the number of object classes and the number of keypoints, respectively.

In the *keypoint regression branch*, each element of a grid cell vector is responsible for predicting the coordinate of a fix-numbered point (see Fig. 2(b)). The position of the centroid point, denoted as $(c_{\mathrm{x}}, c_{\mathrm{y}})$, is obtained by

$$\begin{cases} c_{\mathrm{x}} = \sigma(x_0) + b_{\mathrm{x}} \\ c_{\mathrm{y}} = \sigma(y_0) + b_{\mathrm{y}} \end{cases}, \tag{1}$$

where x_0 and y_0 are the output of the grid cell element 0 and 1, respectively, b_{x} and b_{y} denote the top-left corner coordinates of the grid cell where the centroid point locates, $\sigma(\cdot)$ denotes the sigmoid function: $\sigma(x) = \frac{1}{1+e^{-x}}$. Equation (1) guarantees that the predicted centroid point falls in a grid cell. The position of other keypoints is obtained by adding the output of the remained grid cell elements with the top-left corner coordinates of the grid cell.

In the *keypoint classification branch*, if there are multiple types of MAVs in an image, each type of MAV can be regarded as a class. Then, each MAV can be classified simultaneously with the softmax function: $g(\omega_i) = \frac{e^{\omega_i}}{\sum_{j=1}^{k} e^{\omega_j}}$, where ω_i denotes the output of the grid cell element i, k denotes the number of classes.

In the *keypoint confidence branch*, each grid cell vector contains only one element. The output of the element is used to predict the confidence of a group of keypoints where the centroid point is located in this grid cell.

Loss Function: The loss function is defined as

$$\mathcal{L} = \lambda_{\mathrm{cls}}\mathcal{L}_{\mathrm{cls}} + \lambda_{\mathrm{conf}}\mathcal{L}_{\mathrm{conf}} + \lambda_{\mathrm{reg}}\mathcal{L}_{\mathrm{reg}}, \tag{2}$$

where $\mathcal{L}_{\mathrm{cls}}$, $\mathcal{L}_{\mathrm{reg}}$, and $\mathcal{L}_{\mathrm{conf}}$ denote the loss of keypoint classification, keypoint regression, and keypoint confidence, respectively, λ_{cls}, λ_{reg}, and λ_{conf} are the hyperparameters.

The cross-entropy

$$\mathcal{L}_{\mathrm{cls}} = -\frac{1}{|\mathcal{T}|}\sum_{t_i \in \mathcal{T}}\sum_{j=1}^{C} g_{t_i}^{j} \log(p_{t_i}^{j}) \tag{3}$$

is used to compute the classification loss. In (3), $\mathcal{T}$ and $|\mathcal{T}|$ denote the set of localized MAVs and the size of the set, respectively, C is the number of MAV types, $g_{t_i}^{j}$ and $p_{t_i}^{j}$ denote the groundtruth and the predicted scores for type j, respectively.

The confidence loss is defined as

$$\mathcal{L}_{\mathrm{conf}} = \frac{1}{2}||f(\mathbf{x}^{\mathrm{pr}}, \mathbf{x}^{\mathrm{gt}}) - \sigma(h_{\mathrm{pr}})||_2^2, \tag{4}$$

where h_{pr} denotes the predicted confidence, $\sigma(\cdot)$ denotes the sigmoid function, $f(\mathbf{x})$ is defined as

$$f(\mathbf{x}^{\mathrm{pr}}, \mathbf{x}^{\mathrm{gt}}) = \frac{e^{2\sum_{i=1}^{n}\left(1 - D\left(\mathbf{x}_i^{\mathrm{pr}}, \mathbf{x}_i^{\mathrm{gt}}\right)/D_{\mathrm{thr}}\right)} - 1}{e^2 - 1}. \tag{5}$$

In (5), $\mathbf{x}_i^{\mathrm{pr}}$ and $\mathbf{x}_i^{\mathrm{gt}}$ denote the predicted and groundtruth location of the i-th corner keypoint ($i = 1, \cdots, 8$), $D(\cdot)$ denotes the Euclidean distance. Only the predictions whose distance measured with the groundtruth is within a threshold D_{thr} are considered to compute the confidence loss.

Since the distance between two keypoints in the image is not able to reflect their distance in the 3D space, it could

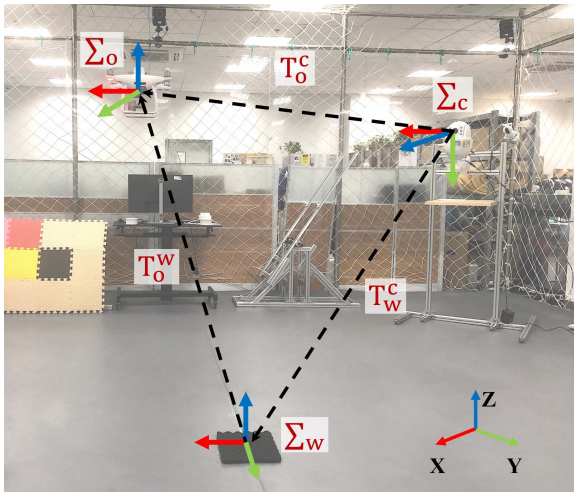


Fig. 4. Coordinate frames of the automatic dataset collection system.



(a) Keypoint prejection with *inaccurate* $\mathbf{T}_{\mathrm{o}}^{\mathrm{c}}$.



(b) Keypoint projection with *accurate* $\mathbf{T}_{\mathrm{o}}^{\mathrm{c}}$.

Fig. 5. An illustration of the importance of accurate calibration of $\mathbf{T}_{\mathrm{o}}^{\mathrm{c}}$.

be adverse to keypoint localization if we only use the image distance to define the keypoint regression loss. To address this problem, a combination loss is introduced. It takes both the 2D regression loss in the image and the 3D regression loss in the space into account. The keypoint regression loss in (2) is defined as

$$\mathcal{L}_{\mathrm{reg}} = \mathcal{L}_{\mathrm{loc}} + \mathcal{L}_{\mathrm{rep}}, \tag{6}$$

where $\mathcal{L}_{\mathrm{loc}}$ and $\mathcal{L}_{\mathrm{rep}}$ denotes the 2D and 3D regression loss, respectively. The 2D regression loss

$$\mathcal{L}_{\mathrm{loc}} = \sum_{i=1}^{n} \frac{1}{2} ||\mathbf{x}_i^{\mathrm{pr}} - \mathbf{x}_i^{\mathrm{gt}}||_2^2 \tag{7}$$

is the mean square error between the predicted keypoint and the groundtruth keypoint on the image. The 3D regression loss

$$\mathcal{L}_{\mathrm{rep}} = \sum_{i=1}^{n} \mathrm{smooth}_{\mathrm{L1}}((\mathbf{I} - \hat{\mathbf{V}}_i)\mathbf{p}_i^{\mathrm{c}}) \tag{8}$$

maps the re-projection error into 3D space [32]. In (8), $\mathbf{I}$ is the identity matrix, $\hat{\mathbf{V}}_i = \mathbf{v}_i\mathbf{v}_i^\top / \mathbf{v}_i^\top\mathbf{v}_i$ is a matrix that projects a 3D point orthogonally to the camera ray, $\mathbf{v}_i$ is the predicted keypoint, $\mathbf{p}_i^{\mathrm{c}}$ is the groundtruth keypoint in the camera coordinate frame, $\mathrm{smooth}_{\mathrm{L1}}(\cdot)$ represents the smooth L1 loss

$$\mathrm{smooth}_{\mathrm{L1}}(x) = \begin{cases} 0.5x^2, & \text{if } |x| < 1, \\ |x| - 0.5, & \text{otherwise.} \end{cases} \tag{9}$$

Pose Estimation: The CPKL-Net has localized the 2D keypoints that are the projections of the pre-defined 3D keypoints. Since we assume the dimension of the target MAV is known (i.e., the keypoints' 3D positions are pre-defined), we can thus estimate the 6D pose of the MAVs with the PnP method based on the correspondences between the 2D and 3D points. In the end, the 3D rotation $\mathbf{R}$ and 3D translation $\mathbf{t}$ from the camera coordinate frame to the MAV coordinate frame can be obtained.

B. Dataset Collection

To train and evaluate the proposed MAV6D method, we next present an automatic dataset collection method.

The setup of the dataset collection is shown in Fig. 4. It includes a MAV, an RGB camera, and a VICON motion capture system. The coordinate frames of VICON, camera, and MAV are respectively denoted as Σ_{w}, Σ_{c}, and Σ_{o}.

Let $\mathbf{T} \in \mathbb{R}^{4\times4}$ be a homogeneous transformation matrix between two frames: $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix}$, where $\mathbf{R} \in \mathbb{R}^{3\times3}$ and $\mathbf{t} \in \mathbb{R}^3$ are rotational and translational transformations, respectively. In particular, $\mathbf{T}_{\mathrm{o}}^{\mathrm{w}}$, $\mathbf{T}_{\mathrm{w}}^{\mathrm{c}}$, and $\mathbf{T}_{\mathrm{o}}^{\mathrm{c}}$ represent the transformations from Σ_{w} to Σ_{o}, Σ_{c} to Σ_{w}, and Σ_{c} to Σ_{o}, respectively.

The most critical step in dataset collection is to accurately calibrate $\mathbf{T}_{\mathrm{o}}^{\mathrm{c}}$, which is the pose of the MAV with respect to the camera. If $\mathbf{T}_{\mathrm{o}}^{\mathrm{c}}$ is inaccurate, the calculated groundtruth of the back-projected 2D keypoints would be too lossy to use as demonstrated in Fig. 5.

Pose Calibration: The object's pose $\mathbf{T}_{\mathrm{o}}^{\mathrm{c}}$ can be computed by

$$\mathbf{T}_{\mathrm{o}}^{\mathrm{c}} = \mathbf{T}_{\mathrm{w}}^{\mathrm{c}}\mathbf{T}_{\mathrm{o}}^{\mathrm{w}}, \tag{10}$$

where $\mathbf{T}_{\mathrm{o}}^{\mathrm{w}}$ can be accurately obtained from VICON and $\mathbf{T}_{\mathrm{w}}^{\mathrm{c}}$ must be calculated. To calculate $\mathbf{T}_{\mathrm{w}}^{\mathrm{c}}$, we minimize the reprojection error

$$\mathbf{T}_{\mathrm{w}}^{\mathrm{c}\,*} = \underset{\mathbf{T}_{\mathrm{w}}^{\mathrm{c}}}{\mathrm{argmin}} \frac{1}{2} \sum_{i=1}^{n} ||\mathbf{p}_i - \frac{1}{s_i}\mathbf{K}\mathbf{T}_{\mathrm{w}}^{\mathrm{c}}\mathbf{p}_i^{\mathrm{w}}||_2^2. \tag{11}$$

where s_i is the depth of point i in the camera coordinate frame, $\mathbf{p}_i = [u_i, v_i, 1]^\top$ is the projected point of a 3D point $\mathbf{p}_i^{\mathrm{w}} = [X_i, Y_i, Z_i, 1]^\top$, $\mathbf{T}_{\mathrm{w}}^{\mathrm{c}} = [\mathbf{R}|\mathbf{t}] \in \mathbb{R}^{3\times4}$ is denoted as an augmented matrix, $\mathbf{K} \in \mathbb{R}^{3\times3}$ is the camera intrinsic matrix, which can be obtained from camera calibration.

To solve (11), the point pairs $\{\mathbf{p}_i^{\mathrm{w}}, \mathbf{p}_i\}$ are needed. By putting the VICON markers on the MAV, their 3D position $\mathbf{p}_i^{\mathrm{w}}$ can be obtained. The 2D projection position $\mathbf{p}_i$ of the markers can be obtained by image processing tools. The $\mathbf{T}_{\mathrm{w}}^{\mathrm{c}}$

can be computed by substituting the point pair set $\{\mathbf{p}_i, \mathbf{p}_i^{\mathrm{w}}\}$ into (11). The detailed process is given in Appendix A.

Keypoint Projection: The nine keypoints of the MAV are defined as shown in Fig. 2(a). After obtaining $\mathbf{T}_{\mathrm{w}}^{\mathrm{c}}$, the keypoints can be projected on the image by

$$\mathbf{p}_i = \frac{1}{s_i}\mathbf{K}\mathbf{T}_{\mathrm{w}}^{\mathrm{c}}\mathbf{T}_{\mathrm{o}}^{\mathrm{w}}\mathbf{p}_i^{\mathrm{o}}, \tag{12}$$

where $\mathbf{p}_i^{\mathrm{o}}$ is the 3D coordinate of keypoint i in Σ_{o}.

Finally, when the MAV flies in the workspace, the 2D keypoints and images can be collected simultaneously. In addition, VICON has a higher sampling frequency (100 Hz) compared with the camera (20 Hz). The timestamp of poses and images are aligned according to the arrival time of images.

IV. Experimental Evaluation of MAV6D

In this section, we first introduce the experimental setup to clarify the evaluation specifics. Then, a series of experiments are conducted to evaluate the performance of the proposed MAV6D. In the end, the ablation studies are carried out to elucidate the efficacy of the key components in MAV6D.

A. Experimental Setup

Datasets for evaluation: With the dataset collection method presented in the last section, we collected an indoor dataset. In this dataset, the DJI Phantom 4 and DJI Mavic 2 are used as the target MAV. It is operated manually and flies at multiple heights with different attitudes. The distance between the MAV and the camera ranges from around 1 m to 6 m. The indoor dataset contains 57,075 images (from 129 videos) with annotated keypoints. The dataset is released at *https://github.com/WestlakeAerialRobotics/MAV6D*.

In addition, to investigate generalization across various objects, we perform experiments on the Occluded-LineMOD dataset [48]. This dataset is an extension of the LineMOD [49] dataset. Each image features multiple annotated objects, and these objects are subjected to extensive occlusion, presenting a significant challenge for pose estimation.

Implementation Details: The parameters of our network are initialized by pre-training on ImageNet [50]. Then, it is re-trained by the experimental dataset. All the experiments are implemented on a computer with Nvidia RTX 2080Ti GPU and Intel(R) Xeon(R) Gold 5118 CPU.

Since we define 9 keypoints in our method, the number of keypoint $kpts$ in the output layer of the network is set to 9. The value of num_cls is set base on the number of object types in the experiment. To balance the performance and inference time of the proposed MAV6D method, the number of convolution kernels C, C', and C'' are set to 448, 128, and 128, respectively.

The values of λ_{cls}, λ_{conf}, and λ_{reg} are set based on multiple experimental tests and we select the value with the best performance. In the early training stage, since the network has a limited localization capability, we only use the 2D regression loss $\mathcal{L}_{\mathrm{loc}}$ as the keypoint regression loss in the first 15 training epochs. When there is only one type of MAV, it does not need to classify the target, and λ_{cls} is set to 0. When a grid cell contains a MAV, λ_{conf} is set to 5; Otherwise, it is set to 1. The value of λ_{reg} is set to 1.

The stochastic gradient descent algorithm is used to optimize the parameters of the network. The momentum is set to be 0.843. The learning rate starts from 0.0032 and decays with a cosine learning rate scheduler $s(i) = \alpha + (1-\alpha)\cos(\pi i/N)$, where i is the current number of training epochs, N is the total number of training epochs, α is the hyperparameter. The N and α are set to 200 and 0.56, respectively. This learning rate scheduler has the effect of starting with a large learning rate that is rapidly decreased to a minimum value before being increased rapidly again. The resetting of the learning rate acts like a simulated restart of the learning process, which re-uses the good weights as the starting point of the restart process.

To avoid overfitting, the keypoint data augmentation (KDA) technique is leveraged in the training. Specifically, the brightness, contrast, hue, and saturation of the images are changed randomly by factors between [0.7, 1.3], [0.8, 1.2], [0.95, 1.05], and [0.5, 1.5], respectively. In addition, the translational and rotational diversity of the 2D keypoints is enhanced. The images are rotated and translated between [-30, +30] degrees and [-10, +10] pixels. To increase the data in the depth dimension, we scale the image by a factor between [0.8, 1.2].

In the experiments, the dataset is split into training and testing sets. Each training set comprises 90% of the samples for training, and the remaining 10% are reserved for validation. For the Phantom 4 in the indoor dataset, there are 11,676 samples (from 59 videos) allocated for training and 5,058 samples (from 9 videos) for testing. As for the Mavic 2, there are 35,379 samples (from 52 videos) for training and 4,962 samples (from 9 videos) for testing. The LineMOD dataset serves as the training set, and the entire Occluded-LineMOD dataset is employed as the testing set.

Evaluation Metrics: We adopt four metrics to evaluate the performance of our method.

The first metric is the average precision (AP). It is typically used to measure the classification and regression performance of the network. The bounding box of each detected object is formed by the maximum and minimum coordinates of each keypoint group in the X and Y directions. The centroid point class is regarded as the object class in the bounding box.

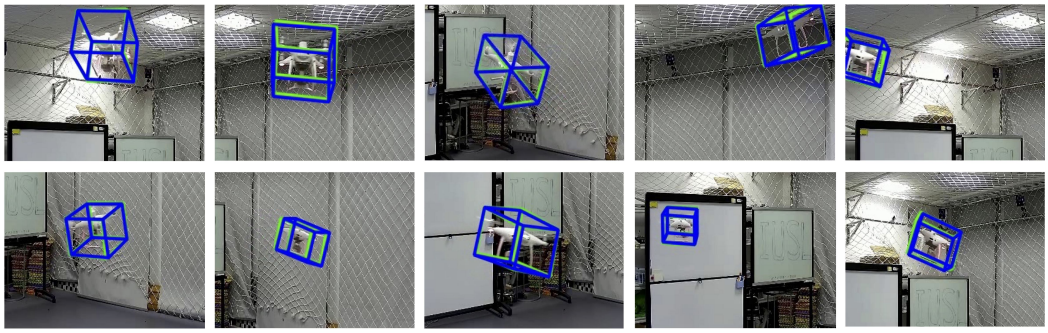
The second metric is ADD. It measures the average 3D distance between the groundtruth and the predicted keypoints. The ADD is defined as

$$\mathrm{ADD} = \frac{1}{|\mathcal{M}|}\sum_{\mathbf{p}\in\mathcal{M}}\left\|(\mathbf{R}\mathbf{p}+\mathbf{t}) - (\tilde{\mathbf{R}}\mathbf{p}+\tilde{\mathbf{t}})\right\|_2^2, \tag{13}$$

where $\mathbf{R}$ and $\mathbf{t}$ denote the groundtruth rotation matrix and translation matrix, respectively, $\tilde{\mathbf{R}}$ and $\tilde{\mathbf{t}}$ denote the predicted ones, $\mathbf{p}$ denotes a keypoint in the keypoint group $\mathcal{M}$. If the ADD of a group of keypoints is less than 50% of the model's diameter, these keypoints are considered to be correct. It is also denoted as ADD(0.5d).

The third metric is $10°10\mathrm{cm}$. It evaluates the performance of the pose estimation method by jointly measuring the rotation error

$$\mathrm{Error}_{\mathrm{rot}} = \arccos\frac{\mathrm{trace}(\mathbf{R}\tilde{\mathbf{R}}^{\top}) - 1}{2}, \tag{14}$$



(a) Estimation results of the Phantom 4.



(b) Estimation results of the Mavic 2.

Fig. 6. Examples of the estimation results by MAV6D on the indoor dataset. The green color 3D bounding box is generated by projecting the predefined keypoints with the groundtruth pose. The blue color 3D bounding box is obtained based on the detected keypoints.

and the translation error

$$\text{Error}_{\text{trans}} = \left\|\mathbf{t} - \tilde{\mathbf{t}}\right\|_2^2. \tag{15}$$

If the rotation error and translation error are less than 10° and $10\ \text{cm}$, respectively, the estimated pose will be regarded to be correct.

The fourth metric is the runtime. Since the estimation method will eventually be implemented in the onboard computer of a MAV, which has limited computing resources, it is important to study the efficiency of the method.

B. Evaluation Results

We demonstrate the estimation results and conduct a comparative analysis with the SOTA methods. Factors influencing the performance of a MAV 6D pose estimation method are investigated. Furthermore, the generalization capability of the proposed method to other objects and the performance of the method when encountering multiple objects are evaluated.

Evaluation on the indoor dataset: The estimation results of the MAV6D method on the indoor dataset are demonstrated in Fig. 6. It can be seen that the MAV6D method has a creditable performance.

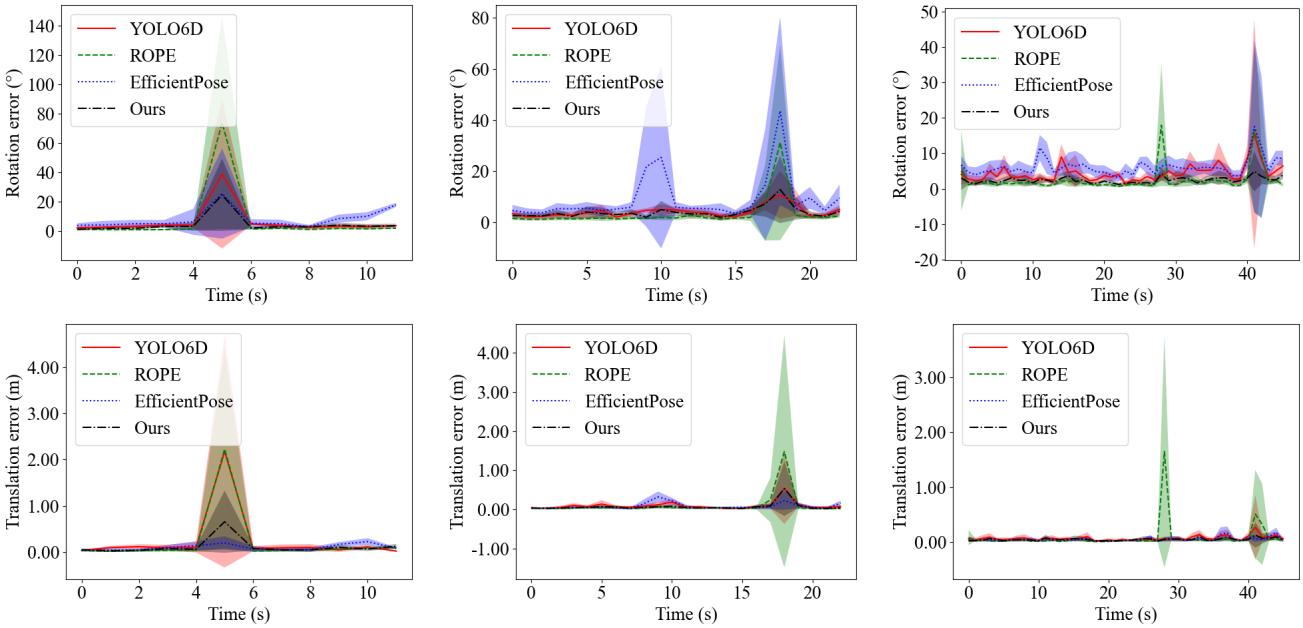
We conduct a comparison between the proposed MAV6D method and the ROPE [26], YOLO6D [27], and EfficientPose [28]. They are all CAD model-free methods. Prior to testing, all the methods are fine-tuned on the training dataset. The evaluation results are reported in Table I. It can be seen that the proposed method achieves the best performance on the metrics of AP, ADD(0.5d), and runtime. In particular, our method is 16 times faster than ROPE. We also tested this method on an embedded computer (Nvidia Jetson Xavier NX). The inference speed could reach at least 63 FPS.

To analyze the factors influencing MAV pose estimation performance, we showcase the performance of the four methods on three testing sequences where the attitude, position, and velocity of the target MAV vary significantly. The mean and standard deviation of the rotation and translation errors are shown in Fig. 7. It can be seen that the performance of the four methods decreases sharply at some times. Through observing the source images, we find that the degradation is caused by truncation and motion blur (see Fig. 8). In particular, the truncation has a great impact on the MAV 6D pose estimation. It causes the estimated results to seriously deviate from the groundtruth.

Evaluation on the Occluded-LineMOD dataset: We adopt the ADD metric to evaluate the accuracy of the estimation result. Since the result of EfficientPose reported in [28] is refined by an additional module, we exclude it from the comparison and select three other SOTA methods (i.e., PVNet [51], Pix2Pose [52], and Hu2 [53]) instead. The evaluation results are illustrated in Table II. It can be seen that the proposed method achieves the best performance on this dataset. In particular, our method outperforms ROPE and PVNet by margins of 21.40% and 11.07% for the "cat" and "duck" objects, respectively. This verifies the effectiveness of the proposed method generalizing on other objects. In addition, although the number of target objects increases to 8, the proposed method maintains superior runtime performance. Its runtime is 13.4 ms which is faster than YOLO6D (30.3 ms), ROPE (181.7 ms), and EfficientPose (190.0 ms).

TABLE I
EVALUATION RESULTS OF DIFFERENT METHODS ON THE INDOOR DATASET

Method	Phantom 4			Mavic 2			Runtime
	AP	10°10cm	ADD(0.5d)	AP	10°10cm	ADD(0.5d)	
YOLO6D	99.67%	71.59%	96.54%	97.45%	44.66%	71.18%	28.5 ms
ROPE	99.23%	**95.41%**	98.24%	97.30%	67.13%	78.53%	178.5 ms
EfficientPose	99.63%	78.27%	97.33%	97.13%	43.25%	71.34%	186.2 ms
Ours	**99.72%**	86.75%	**98.97%**	**97.50%**	**71.39%**	**85.53%**	**10.9 ms**



(a) Image sequence 1 (b) Image sequence 2 (c) Image sequence 3

Fig. 7. The estimation errors of the MAV6D method on three image sequences. The line and the color range represent the mean and standard deviation of the estimation error in each second, respectively.

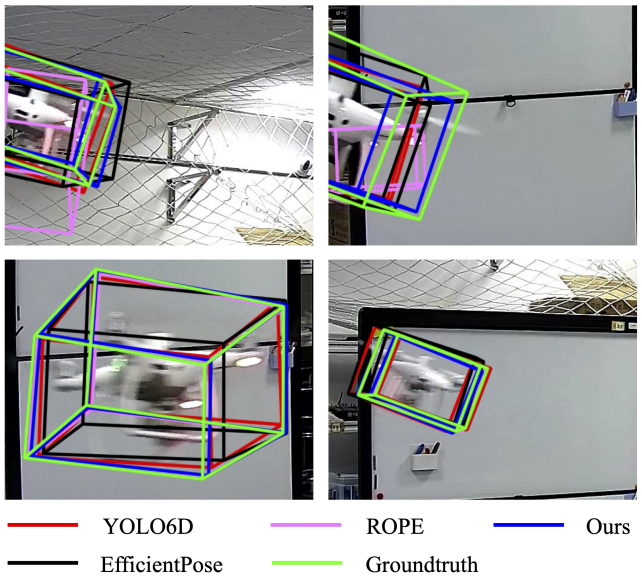


Fig. 8. Estimation results in scenarios involving motion blur and truncation. The first-row images depict cases with truncation, where the MAV is situated at the edge of the camera view. The second-row images illustrate cases with motion blur.

C. Ablation Study

To evaluate the impact of KDA, $\mathcal{L}_{\text{rep}}$, and MFMF on the method's performance, we perform experiments by introducing each of the three components one by one into the baseline method. The method that does not have the three components is adopted as the baseline.

The experimental results are illustrated in Table III. It can be seen that each of the three components is capable of improving the performance of a MAV 6D pose estimation method. In particular, with $\mathcal{L}_{\text{rep}}$, the performance is significantly enhanced. Specifically, the $10°10\text{cm}$ is increased more than 10% compared with the baseline. In addition, the method incorporating KDA or MFMF components demonstrates an improvement of over 8% in ADD(0.5d).

In addition, we investigate the MAV 6D pose estimation method's performance when trained and tested on different types of MAVs. Initially, we train the proposed MAV6D using the Phantom 4 training set and evaluate its performance on the Mavic 2 testing set. Then, we fine-tune the MAV6D specifically with the Mavic 2 training set and re-evaluate its performance on the testing set.

TABLE II
EVALUATION RESULTS ON THE OCCLUDED-LINEMOD DATASET USING ADD METRIC (%)

Method	ape	can	cat	driller	duck	eggbox	glue	holepuncher	Average
YOLO6D	2.48	17.48	0.67	7.66	1.14	-	10.08	5.45	6.42
ROPE	28.03	75.06	25.53	61.86	19.07	45.62	**56.92**	55.54	45.95
PVNet	15.81	63.30	16.68	65.65	25.24	50.17	49.62	39.67	40.77
Pix2Pose	22.00	44.70	22.70	44.70	15.00	25.20	32.40	49.50	32.03
Hu2	19.20	65.10	18.90	69.00	25.30	**52.00**	51.40	45.60	43.30
Ours	**36.15**	**80.20**	**46.93**	**75.70**	**36.31**	45.38	49.30	**56.61**	**53.32**

TABLE III
ABLATION STUDY OF THE PROPOSED MAV6D METHOD

Method				Evaluation result (%)		
	KDA	$\mathcal{L}_{\text{rep}}$	MFMF	AP	10°10cm	ADD(0.5d)
Baseline	–	–	–	99.60	69.41	87.98
	+	–	–	99.64	75.17	96.90
	–	+	–	**99.80**	**81.38**	**97.33**
	–	–	+	99.74	73.23	95.31

TABLE IV
PERFORMANCE COMPARISON OF THE METHOD USING DIFFERENT MAVS IN THE TRAINING AND TESTING PHASES (%)

Testing MAV	Training MAV	AP	10°10cm	ADD(0.5d)
Mavic 2	Phantom 4	8.67	0.06	1.29
	Mavic 2	**97.50**	**71.39**	**85.53**

The experimental results are presented in Table IV. It can be seen that the MAV6D trained on a single type of MAV exhibits limited performance on another type. However, after fine-tuning on the target MAV type, its performance is significantly enhanced. Thus, with fine-tuning, the proposed MAV6D is capable of estimating different types of MAVs with creditable estimation performance.

To summarize this section, the MAV6D method exhibits high performance on the indoor dataset and the Occluded-LineMOD dataset, maintaining superior runtime even with an increased number of target objects. The method demonstrates effectiveness in generalizing to other objects, and the introduced components (i.e., KDA, $\mathcal{L}_{\text{rep}}$, and MFMF) significantly improve MAV 6D pose estimation. Through fine-tuning, the MAV6D can estimate various types of MAVs with creditable performance. In addition, challenges such as truncation and motion blur impact the MAV 6D pose estimation task.

V. DOMAIN ADAPTION FOR MAV 6D POSE ESTIMATION

MAV swarming tasks are usually performed in outdoor environments. However, the method trained on the indoor dataset is not directly applicable to outdoor environments. To address this problem, we propose a self-training-based UDA method to enable the MAV6D method to work effectively across different domains.

A. The Proposed UDA Method

This UDA method consists of two steps. The first step is pseudo-label generation, which utilizes the pre-trained network to generate pseudo-labels for the unlabeled data of the target domain. The second step is network re-training, which re-trains the network with the pseudo-labeled data of the target domain.

Some notions are defined first. Let $\mathbb{D}_{\text{s}} = \left\{(\mathbf{I}_{\text{s}}^i, \mathbf{Y}_{\text{s}}^i)\right\}|_{i=1}^{N_{\text{s}}}$ be N_{s} labeled images of the source domain, where $\mathbf{I}^i$ and $\mathbf{Y}^i$ represent the i-th image and its annotation (i.e., keypoints), respectively. The dataset of the target domain is denoted as $\mathbb{D}_{\text{t}} = \left\{(\mathbf{I}_{\text{t}}^i)\right\}|_{i=1}^{N_{\text{t}}}$, which contains N_{t} images without groundtruth annotations. The parameter of the network is denoted as Θ.

Pseudo-label Generation: Since the network trained on the source domain fails to localize keypoints in the target domain, we cannot generate the pseudo-labels directly. To tackle this problem, we construct a synthetic dataset, which is obtained by cropping the MAVs in the source domain data and pasting it to the target domain data. In this way, the domain gap is narrowed by training the network with the synthetic dataset.

The data synthesis operation is defined as

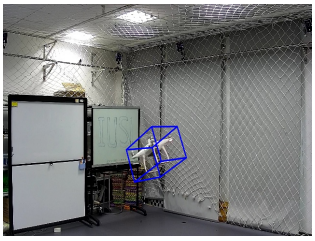
$$\mathbf{I}_{\text{sy}} = w(\mathbf{I}_{\text{s}}, \mathbf{b}) \circledcirc \bar{\mathbf{I}}_{\text{t}}, \tag{16}$$

where $\mathbf{b}$ denotes the area containing all the keypoints in image $\mathbf{I}_{\text{s}}$, $w(\cdot)$ denotes the crop operation, $\circledcirc$ denotes the paste operation, $\bar{\mathbf{I}}_{\text{t}}$ denotes the background image of the target domain, which can be obtained by capturing the outdoor scenes or masking the MAV in the image of the target domain, $\mathbf{I}_{\text{sy}}$ denotes the synthetic image. The operation process is illustrated in Fig. 9. Since the MAV is cropped from the source domain data, the annotations of the synthetic dataset are the same as the source domain dataset. We denote the synthetic dataset as $\mathbb{D}_{\text{sy}} = \left\{(\mathbf{I}_{\text{sy}}^i, \mathbf{Y}_{\text{s}}^i)\right\}|_{i=1}^{N_{\text{s}}}$.

To produce the pseudo-labels for target domain data, the network is trained with $\mathbb{D}_{\text{s}}$ and $\mathbb{D}_{\text{sy}}$. The learning objective is written as

$$\underset{\Theta}{\operatorname{argmin}} \frac{1}{N_{\text{s}}} \left[\sum_{\mathbf{I}_{\text{s}} \in \mathbb{D}_{\text{s}}} \alpha \mathcal{L}(\mathbf{I}_{\text{s}}, \mathbf{Y}_{\text{s}}) + \sum_{\mathbf{I}_{\text{sy}} \in \mathbb{D}_{\text{sy}}} \beta \mathcal{L}(\mathbf{I}_{\text{sy}}, \mathbf{Y}_{\text{s}}) \right], \tag{17}$$

where $\mathcal{L}(\cdot)$ is defined in (2), α and β are hyper-parameters. The trained network is then leveraged to predict keypoints in the target domain data. The keypoints with high confidence



(a) Source domain image.



(b) Target domain background image.



(c) Mixed domain image.

Fig. 9. Illustration of mixed domain image generation. In (a), the source domain image is with labels. In (c), the mixed domain image has the same labels as the source domain image.

(greater than a threshold ξ) are selected, and their x-y coordinates and classes are treated as pseudo-labels.

Since the output of the pre-trained network is not necessarily reliable, we adopt an automatic validation technique to eliminate the pseudo-label predictions with high confidence but false positives. First, we calculate the Euclidean distance between the keypoints of the target domain sample and the keypoints of all samples in the source domain dataset and select the closest sample. Then, the hash similarity [54] between the regions of the two samples surrounded by keypoints is calculated. By establishing a similarity threshold, pseudo-labels with high confidence but false positives are automatically filtered out. With the finally selected $N_{\mathrm{t}}^{'}$ images and their corresponding pseudo-labels, we denote the pseudo-labeled data of the target domain as $\mathbb{D}_{\mathrm{t}}^{'} = \left\{(\mathbf{I}_{\mathrm{t}}^{i}, \tilde{\mathbf{Y}}_{\mathrm{t}}^{i})\right\}|_{i=1}^{N_{\mathrm{t}}^{'}}$.

Network Re-training: The network is trained again from scratch using the generated pseudo-labels for an additional keypoint regression loss on the target-domain training set. The optimization problem is

$$\underset{\Theta}{\operatorname{argmin}} \frac{1}{N_{\mathrm{s}}} \left[\sum_{\mathbf{I}_{\mathrm{s}} \in \mathbb{D}_{\mathrm{s}}} \alpha \mathcal{L}(\mathbf{I}_{\mathrm{s}}, \mathbf{Y}_{\mathrm{s}}) + \sum_{\mathbf{I}_{\mathrm{sy}} \in \mathbb{D}_{\mathrm{sy}}} \beta \mathcal{L}(\mathbf{I}_{\mathrm{sy}}, \mathbf{Y}_{\mathrm{s}}) \right] + \frac{1}{N_{\mathrm{t}}^{'}} \sum_{\mathbf{I}_{\mathrm{t}} \in \mathbb{D}_{\mathrm{t}}^{'}} \gamma \mathcal{L}(\mathbf{I}_{\mathrm{t}}, \tilde{\mathbf{Y}}_{\mathrm{t}}), \quad (18)$$

where γ is a hyper-parameter. In practice, we set α, β, and γ to 1 for simplicity.

To obtain a network with higher performance, the pseudo-labels are re-generated with the re-trained network, then the step of network re-training is operated with the selected pseudo-labels again. This process is repeated until the performance of the network is satisfactory. Note that, the high-quality pseudo-labels obtained each time will be retained, and we do not re-generate new pseudo-labels for the data that already have pseudo-labels.

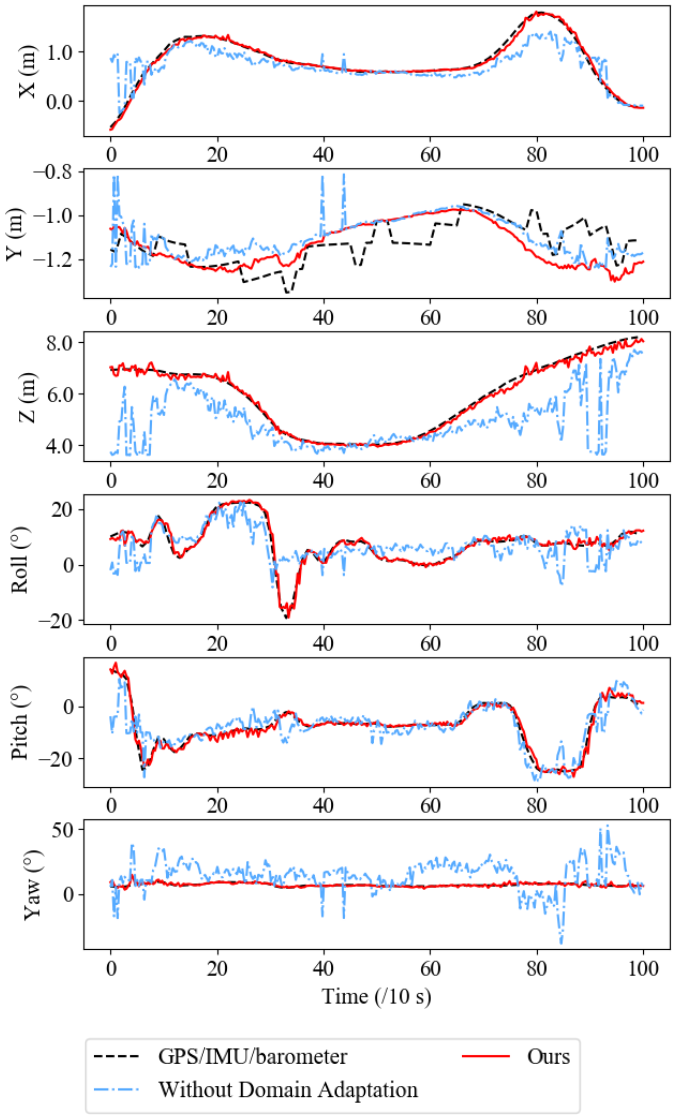


Fig. 10. Performance comparison of the MAV6D method processed with or without the proposed UDA method.

B. Experimental Setup

Outdoor MAV 6D Pose Dataset: The image sequences in the outdoor dataset are collected using a DJI Phantom4 (target) and a DJI M300 (observer), which is equipped with a DJI H20T camera. The 6D poses are obtained by the sensors installed on the MAVs such as the barometer, the global positioning system (GPS) receiver, and the inertial measurement unit (IMU).

This dataset is composed of two parts. The first part is a dataset for quantitative experiments. This part of the data contains 300 images as well as the corresponding poses. The roll, pitch, and yaw of the target MAV vary in $[-19.3°, +22.3°]$, $[-25.4°, +14.1°]$, and $[+4.0°, +8.2°]$, respectively. The distance between the target MAV and the camera changes from 4.02 m to 8.20 m. The second part is a dataset collected for

qualitative experiments. The image sequences in this part have different characteristics such as varying lighting intensities, weather conditions, observing distances, and backgrounds.

Implementation Details: In the pseudo-label generation, the CPKL-Net pre-trained on the indoor dataset is adopted to train with the synthetic dataset, which is processed with data augmentation. The whole training process lasts for 200 epochs. The last checkpoint is used to generate the pseudo-labels. In the network re-training, the network is re-trained with (18). To obtain higher quality pseudo-labels, the re-trained network is adopted to yield new pseudo-labels. In addition, since the number of high-quality pseudo-labels influences the performance of the MAV6D method, we repeat the steps of pseudo-label generation and network re-training three times. With the increase of the repeated round, the number of pseudo-labels also increases. In all experiments, the training settings are the same as training on MAV6D (see Section IV-A).

The other details of experiments and evaluations are given below. In the experiment of estimating the MAV 6D pose using the static camera, one background image is used to generate the synthetic dataset. While in the experiments using the moving camera, two to four background images, which are with low overlap rates, are adopted. In addition, in the quantitative analysis, since the sampling frequency of the camera (30 Hz) is different from the data recording frequency of the MAV (10 Hz), the averaged estimation results in every three frames are compared with the groundtruth poses for error statistics and analysis. In the qualitative experiments, the experimental results are illustrated by visualizing the keypoints on the image.

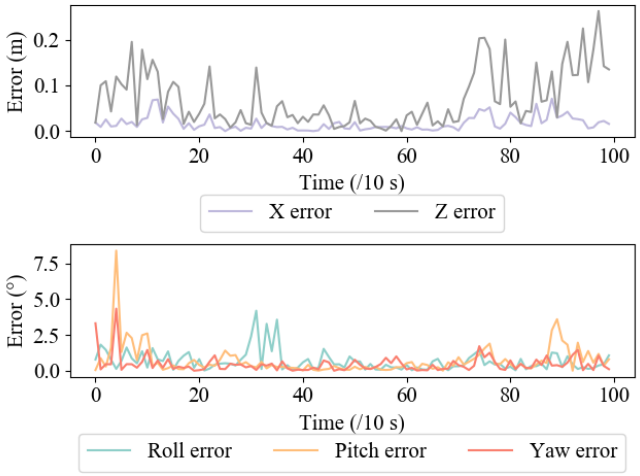
C. Evaluation Results

In this part, we first evaluate the performance of the proposed UDA method with quantitative experiments. Then, qualitative experiments are conducted to evaluate the potential real-world challenges in the outdoor environment influencing the performance of the MAV6D method.

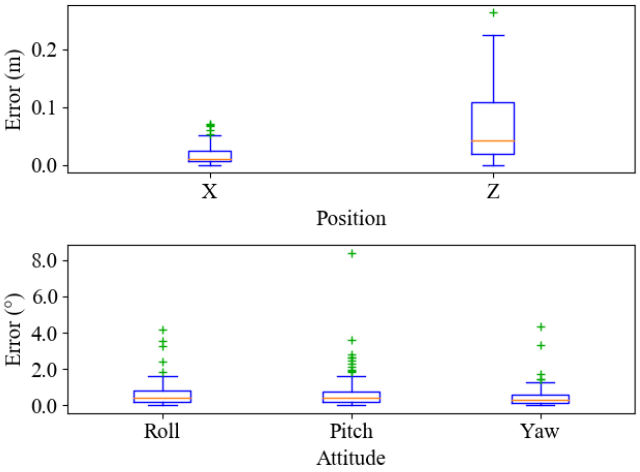
Quantitative Results and Analysis: Considering that there are few similar works in the UDA for object 6D pose estimation, we evaluate the proposed UDA method by making a comparison of the estimated results with the measurements obtained from GPS/IMU/barometer of the target MAV. Since the height of MAV measured by the sensors has limited accuracy, we do not evaluate the estimation result in the Y direction. To make a detailed comparison, we decouple the MAV 6D pose into position and attitude.

The experimental results are demonstrated in Fig. 10. As can be seen, the estimated position and attitude achieve a comparable result with the measurements. In addition, comparing our results with the version without domain adaptation, it can be seen that domain adaptation can significantly improve the performance of the MAV6D method on the outdoor dataset. This verifies the effectiveness of the proposed UDA method and the importance of pseudo-labels in improving model performance in the target domain.

The pose estimation errors are counted in Fig. 11. It can be seen that the maximum estimation error in the Z direction



(a) Position and attitude errors.



(b) Boxplot of error statistics.

Fig. 11. Pose estimation errors. The position and attitude estimation errors are demonstrated in (a). The statistics for each error are demonstrated in (b).



Fig. 12. The estimation results of the MAV6D method when encountering two MAVs in the outdoor environment. The red and blue boxes denote the estimation results of Mavic 2 and Phantom 4, respectively.

is about 0.25 m and the mean errors of roll, pitch, and yaw are below 1.50°. In addition, the statistics reveal that the estimation accuracy in the X direction exhibits lower uncertainty compared to the Z direction. The roll, pitch, and yaw estimation results demonstrate stability and high accuracy.

Qualitative Results and Analysis: The estimation results of the MAV6D method when encountering multiple MAVs in the outdoor environment are illustrated in Fig. 12. It can be seen that after being processed with the proposed UDA method, the MAV6D method is capable of estimating multiple MAVs simultaneously in the target domain.

The qualitative results on the image sequence captured by a



(a) The estimation results on dynamic backgrounds.



(b) The estimation results for a MAV with a large maneuver.

Fig. 13. Qualitative results on the images captured by a dynamic camera.



Fig. 14. The estimation results under partial occlusion. The images are arranged chronologically from left to right.

dynamic camera are shown in Fig. 13. As can be seen, with the proposed UDA method, the MAV6D method is workable in dynamic environments even though there are wide variations of the background. We also test the case of MAV moving with a large maneuver. It can be seen that the MAV6D method can still obtain satisfactory results.

The impact of partial occlusion caused by potential obstacles on the MAV6D's performance is shown in Fig. 14. It can be seen that when a minor portion of the MAV's body is occluded, the estimated results exhibit only a slight deviation. However, when the area of the occluded part surpasses 50%, the performance of the MAV6D method gradually deteriorates.

To investigate the impact of lighting conditions on the performance of the MAV6D method, we conduct experiments under three different lighting intensities. The evaluation results are illustrated in Fig. 15. It can be seen that the MAV6D method exhibits superior performance when the lighting intensity is above 2200 LUX. However, as the lighting intensity decreases to 154 LUX, the MAV features in the image become challenging to distinguish, resulting in a decline in MAV6D performance.

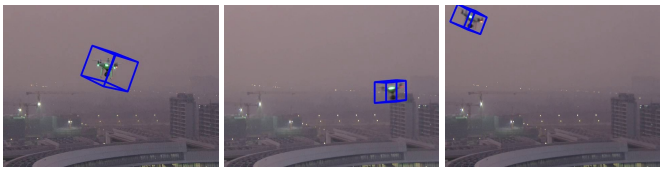
We conduct experiments at various distances between the target MAV and the observing camera, specifically at distances of around 8.0 m, 20.0 m, and 30.7 m. The experimental results are illustrated in Fig. 16. Within a range of around 20.0 m, the MAV6D method exhibits the capability to estimate the MAV's 6D pose with minor deviations. However, when



(a) Lighting intensity: 6900 LUX.



(b) Lighting intensity: 2200 LUX.



(c) Lighting intensity: 154 LUX.

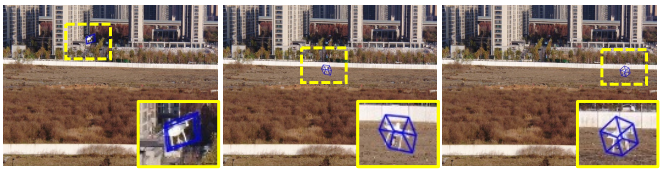
Fig. 15. Estimate the MAV's 6D pose under varying lighting intensities.



(a) Distance: around 8.0 m.



(b) Distance: around 20.0 m.



(c) Distance: around 30.7 m.

Fig. 16. Estimate the target MAV's 6D pose at various distances. The blue 3D bounding box is generated by the detected keypoints. In (b) and (c), the yellow solid line box provides an enlarged view of the dotted line box.



Fig. 17. The estimation results of the images captured with 20 times focal length. The distance between the camera and the MAV is more than 30 m.

the distance extends to about 30.7 m, the estimation results degrade. To enhance the effective working distance of the MAV6D method, one strategy is to adjust the camera's focal length. As illustrated in Fig. 17, when the focal length is adjusted to 20 times, the effective working distance of the MAV6D method can be extended to over 30 m.

In summary, we introduce a UDA method to enhance the adaptability of the MAV6D method to diverse environments in this section. Extensive quantitative and qualitative experiments verify the effectiveness of the proposed UDA method. In addition, the impact of various real-world potential challenges on the MAV6D method's performance is evaluated, including the number of MAVs, lighting conditions, obstacles, dynamic backgrounds, and observing distances.

VI. Application to Target Motion Tracking

In this section, we apply the pose estimated by MAV6D to motion estimation. We will show that without the estimated 3D attitude information, a Kalman filter (KF) with a linear constant state-transition model has a considerable estimation lag. The lag increases when the target MAV flies agilely. By contrast, with the 3D attitude information, we can use a Kalman filter with a nonlinear state-transition model to significantly reduce the estimation lag.

A. Motion Tracking Method

In this part, the process model, the measurement model, and the Kalman filter for motion tracking of the target MAV are presented.

Process Model: Let $\mathbf{p}_\mathrm{w} = [p_{\mathrm{w,x}}, p_{\mathrm{w,y}}, p_{\mathrm{w,z}}]^\top \in \mathbb{R}^3$ and $\mathbf{v}_\mathrm{w} = [v_{\mathrm{w,x}}, v_{\mathrm{w,y}}, v_{\mathrm{w,z}}]^\top \in \mathbb{R}^3$ respectively be the position and velocity of the target MAV in the world coordinate frame. The attitude represented by Euler angles (roll, pitch, yaw) is denoted as $\boldsymbol{\zeta} = [\phi, \theta, \psi]^\top$. Let $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$ be the column vectors of the 3×3 identity matrix. The kinematic model of the target MAV is

$$\begin{bmatrix}\dot{\mathbf{p}}_\mathrm{w}\\ \dot{\mathbf{v}}_\mathrm{w}\end{bmatrix} = \begin{bmatrix}\mathbf{v}_\mathrm{w}\\ -\mathbf{R}_\mathrm{o}^\mathrm{w}\mathbf{e}_3 a_\mathrm{o} + \mathrm{g}\mathbf{e}_3\end{bmatrix}, \tag{19}$$

where a_o denotes the acceleration of the target MAV in the MAV coordinate frame, g is the local gravitational acceleration, the rotation matrix $\mathbf{R}_\mathrm{o}^\mathrm{w}$ is

$$\mathbf{R}_\mathrm{o}^\mathrm{w} = \begin{bmatrix} s_\theta c_\psi & s_\theta c_\psi s_\phi - s_\psi c_\phi & s_\theta c_\psi c_\phi + s_\psi s_\phi \\ c_\theta s_\psi & s_\theta s_\psi s_\phi + c_\psi c_\phi & s_\theta s_\psi c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix},$$

where $s* = \sin(*), c* = \cos(*)$. The estimated MAV attitude from the MAV6D method is substituted into (19).

Since we do not know the thrust force of the target MAV, we assume that the magnitude of the thrust is equal to its gravity plus zero-mean white noise w. Then, the nonlinear process model of the target MAV can be written as

$$\begin{bmatrix}\dot{\mathbf{p}}_\mathrm{w}\\ \dot{\mathbf{v}}_\mathrm{w}\end{bmatrix} = \begin{bmatrix}\mathbf{v}_\mathrm{w}\\ -(\mathrm{g}+w)\mathbf{R}_\mathrm{o}^\mathrm{w}\mathbf{e}_3 + \mathrm{g}\mathbf{e}_3\end{bmatrix}. \tag{20}$$

The process model can be rewritten in a compact form as

$$\dot{\mathbf{x}} = f(\mathbf{x}, w), \tag{21}$$

where

$$\mathbf{x} = \begin{bmatrix}\mathbf{p}_\mathrm{w}\\ \mathbf{v}_\mathrm{w}\end{bmatrix}, \quad f(\mathbf{x}, w) = \begin{bmatrix}\mathbf{v}_\mathrm{w}\\ -(\mathrm{g}+w)\mathbf{R}_\mathrm{o}^\mathrm{w}\mathbf{e}_3 + \mathrm{g}\mathbf{e}_3\end{bmatrix}.$$

Let

$$\mathbf{A} = \frac{\partial f}{\partial \mathbf{x}}, \quad \mathbf{B} = \frac{\partial f}{\partial w},$$

be the Jacobians of $f(\mathbf{x}, w)$ with respect to $\mathbf{x}$ and w. It can be calculated that

$$\mathbf{A} = \begin{bmatrix}\mathbf{0}_{3\times3} & \mathbf{I}_{3\times3}\\ \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3}\end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix}\mathbf{0}_{3\times1}\\ -s_\theta\\ c_\theta s_\phi\\ c_\theta c_\phi\end{bmatrix}.$$

Measurement Model: The measurement is the estimated target's position $\mathbf{p}_\mathrm{w}$ from the MAV6D method. Hence, the observation model of the estimation is written as

$$\mathbf{z} = \mathbf{p}_\mathrm{w} = \mathbf{H}\mathbf{x} + \boldsymbol{\nu}, \tag{22}$$

where $\mathbf{H} = [\mathbf{I}_{3\times3}, \mathbf{0}_{3\times3}]$ is the measurement matrix, $\boldsymbol{\nu}$ is the observation noise.

Pose-Kalman Filter for Target MAV Motion Tracking: The Kalman filter to track the target motion with the process model (21) and the observation model (22), called Pose Kalman filter (Pose-KF), is given below.

Prediction:

$$\begin{aligned}\mathbf{F} &= \mathbf{I}_{6\times6} + \Delta t\mathbf{A},\\ \boldsymbol{\Gamma}_{k+1} &= \mathbb{1}_{6\times1} + \Delta t\mathbf{B}_{k+1},\\ \hat{\mathbf{x}}_{k+1}^- &= \hat{\mathbf{x}}_k + \Delta t f(\hat{\mathbf{x}}_k, 0),\\ \hat{\mathbf{P}}_{k+1}^- &= \mathbf{F}\hat{\mathbf{P}}_k\mathbf{F}^\top + \boldsymbol{\Gamma}_{k+1}\mathbf{Q}\boldsymbol{\Gamma}_{k+1}^\top.\end{aligned} \tag{23}$$

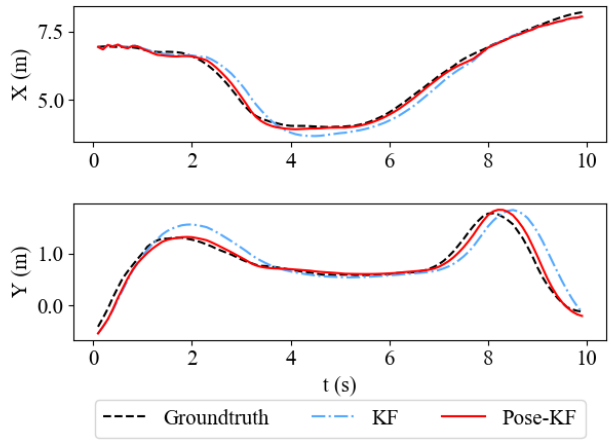
Correction:

$$\begin{aligned}\mathbf{K}_{k+1} &= \hat{\mathbf{P}}_{k+1}^-\mathbf{H}^\top\left(\mathbf{R}_{\boldsymbol{\nu}} + \mathbf{H}\hat{\mathbf{P}}_{k+1}^-\mathbf{H}^\top\right),\\ \hat{\mathbf{x}}_{k+1} &= \hat{\mathbf{x}}_{k+1}^- + \mathbf{K}_{k+1}\left(\mathbf{z}_{k+1} - \mathbf{H}\hat{\mathbf{x}}_{k+1}^-\right),\\ \hat{\mathbf{P}}_{k+1} &= \hat{\mathbf{P}}_{k+1}^- - \mathbf{K}_{k+1}\mathbf{H}\hat{\mathbf{P}}_{k+1}^-.\end{aligned} \tag{24}$$

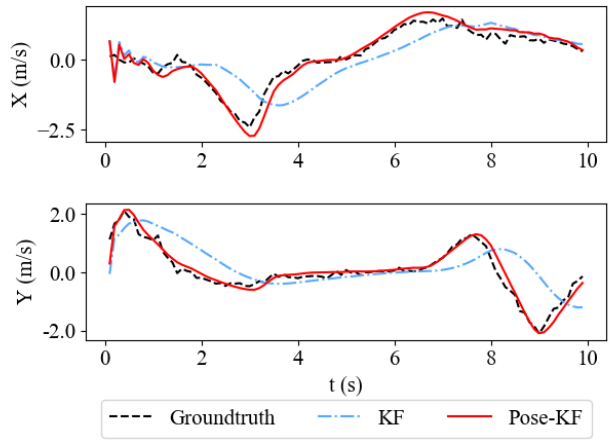
Here, Δt is the sampling time, $\hat{\mathbf{x}}_{k+1}^-$ and $\hat{\mathbf{P}}_{k+1}^-$ are the prior state estimate and covariance of the estimation error, respectively, $\hat{\mathbf{x}}_{k+1}$ and $\hat{\mathbf{P}}_{k+1}$ are the posterior state estimate and covariance of the estimation error, respectively, $\mathbf{K}_{k+1}$ is the Kalman gain matrix at time $k+1$, $\mathbf{Q} = \sigma_q\mathbf{I}_{6\times6}$ and $\mathbf{R}_{\boldsymbol{\nu}} = \sigma_r\mathbf{I}_{3\times3}$ are the covariance matrices of the system error and the measurement error, respectively. The Pose-KF starts with initial estimated state $\hat{\mathbf{x}}_0 \in \mathbb{R}^{6\times1}$ and the estimated covariance matrix $\hat{\mathbf{P}}_0 = \mathbf{I}_{6\times6} \in \mathbb{R}^{6\times6}$.

B. Experimental Results

The Pose-KF method is executed with a frequency of 10 Hz. The MAV 6D pose obtained in Section V-C is used in the target motion tracking. To realize the Kalman filter, the obtained MAV 6D poses are transformed to the world coordinate frame first. The attitude information is used in correcting the process model as the true value and the 3D position is used as the measurement input of the Pose-KF. The σ_q, σ_r, and Δt of the Pose-KF are set to 0.01, 0.1, and 0.1 s, respectively. In addition, to evaluate the performance of the method, the root



(a) Position.



(b) Velocity.

Fig. 18. The lag in the velocity estimation can be significantly reduced when the 3D attitude information is properly utilized.

TABLE V
EXPERIMENTAL RESULTS

Methods	RMSE-$\mathbf{p}$ (m)	RMSE-$\mathbf{v}$ (m/s)
KF	0.3461	1.6991
Pose-KF	0.1410	1.1522

mean square error (RMSE) is adopted. The position RMSE is defined as RMSE-$\mathbf{p}$ $= \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left\|\mathbf{p}_i^{\mathrm{es}} - \mathbf{p}_i^{\mathrm{gt}}\right\|_2^2}$, where $\mathbf{p}_i^{\mathrm{es}}$ and $\mathbf{p}_i^{\mathrm{gt}}$ denote the estimated and the groundtruth position, respectively. The velocity RMSE is defined as RMSE-$\mathbf{v}$ $= \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left\|\mathbf{v}_i^{\mathrm{es}} - \mathbf{v}_i^{\mathrm{gt}}\right\|_2^2}$, where $\mathbf{v}_i^{\mathrm{es}}$ and $\mathbf{v}_i^{\mathrm{gt}}$ denote the estimated and the groundtruth velocity, respectively.

The experimental results are given in Fig. 18 and Table V. As can be seen, the traditional KF method has a large lag when tracking the target, especially when the velocity of the target changes drastically. By contrast, the Pose-KF method, incorporating pose information into a precise MAV motion model, has a faster response and high accuracy in estimating target MAV velocity.

In conclusion, by introducing the MAV 6D pose, the Kalman filter can significantly improve the accuracy of the state-transition model and the response speed of the filter in estimating higher-order information. This is beneficial to track highly maneuverable targets in the 3D space.

VII. CONCLUSION

This paper presented an efficient MAV 6D pose estimation method, named MAV6D. To train the MAV6D method, a high-quality indoor dataset was collected with our developed automatic dataset collection method. To enable the MAV6D method to work effectively in outdoor environments, we proposed a self-training-based UDA method. Extensive experiments verified the effectiveness of the proposed MAV6D method and UDA method. Finally, we showed that attitude information can significantly improve state estimation accuracy.

In the future, the problem needs to be further studied. The MAV studied in this work is known in advance. However, it may be unknown in some applications. Thus, it is important to extend the 6D pose estimation method to unknown MAVs.

APPENDIX

A. Pose Calibration

The objective function of minimizing reprojection error can be written as

$$\mathbf{T}_{\mathrm{w}}^{\mathrm{c}\,*} = \underset{\mathbf{T}_{\mathrm{w}}^{\mathrm{c}}}{\operatorname{argmin}} \frac{1}{2}\sum_{i=1}^{n} ||\mathbf{p}_i - \frac{1}{s_i}\mathbf{K}\mathbf{T}_{\mathrm{w}}^{\mathrm{c}}\mathbf{p}_i^{\mathrm{w}}||_2^2. \tag{25}$$

Since $\mathbf{T}_{\mathrm{w}}^{\mathrm{c}} \in \mathbf{SE}(3)$, (25) can not be differentiated. To address this problem, we adopt the exponential map operator that connects Lie algebra $\mathbf{se}(3)$ to Lie group $\mathbf{SE}(3)$. Then, (25) can be rewritten as

$$\boldsymbol{\xi}^* = \underset{\boldsymbol{\xi}}{\operatorname{argmin}} \frac{1}{2}\sum_{i=1}^{n} ||\mathbf{p}_i - \frac{1}{s_i}\mathbf{K}\exp(\boldsymbol{\xi}^{\wedge})\mathbf{p}_i^{\mathrm{w}}||_2^2, \tag{26}$$

where $\boldsymbol{\xi} = \begin{bmatrix}\boldsymbol{\rho}\\ \boldsymbol{\phi}\end{bmatrix} \in \mathbb{R}^6$,$\boldsymbol{\rho} \in \mathbb{R}^3$, $\boldsymbol{\phi} \in \mathbb{R}^3$,$\boldsymbol{\xi}^{\wedge} = \begin{bmatrix}\boldsymbol{\phi}^{\wedge} & \boldsymbol{\rho}\\ \mathbf{0}^{\mathrm{T}} & 0\end{bmatrix} \in \mathbb{R}^{4\times4}$,$\boldsymbol{\phi}^{\wedge} \in \mathbb{R}^{3\times3}$.

The reprojection error of one point in (26) can be written as

$$\boldsymbol{e} = \mathbf{p} - \frac{1}{s}\mathbf{K}\exp(\boldsymbol{\xi}^{\wedge})\mathbf{p}^{\mathrm{w}}. \tag{27}$$

We make a first-order Taylor expansion at this point. Then, the Jacobian matrix of (27) can be obtained with the Lie algebra perturbation model

$$\boldsymbol{J} = \lim_{\delta\boldsymbol{\xi}\to0}\frac{e(\delta\boldsymbol{\xi}\oplus\boldsymbol{\xi})}{\delta\boldsymbol{\xi}} = \frac{\partial\boldsymbol{e}}{\partial\mathbf{p}^{\mathrm{c}}}\frac{\partial\mathbf{p}^{\mathrm{c}}}{\partial\delta\boldsymbol{\xi}}, \tag{28}$$

where $\mathbf{p}^{\mathrm{c}} = \exp(\boldsymbol{\xi}^{\wedge})\mathbf{p}^{\mathrm{w}} = [X^{\mathrm{c}}, Y^{\mathrm{c}}, Z^{\mathrm{c}}]^{\top}$ is the coordinate of point $\mathbf{p}$ at camera coordinate frame, $\oplus$ denotes the left-tangent-space perturbation. In addition, by solving $\frac{\partial\boldsymbol{e}}{\partial\mathbf{p}^{\mathrm{c}}}$ and $\frac{\partial\mathbf{p}^{\mathrm{c}}}{\partial\delta\boldsymbol{\xi}}$, (28) can be represented as

$$\boldsymbol{J} = \begin{bmatrix} -\frac{f_{\mathrm{x}}}{Z^{\mathrm{c}}} & 0 & \frac{f_{\mathrm{x}}X^{\mathrm{c}}}{Z^{\mathrm{c}2}} & \frac{f_{\mathrm{x}}X^{\mathrm{c}}Y^{\mathrm{c}}}{Z^{\mathrm{c}2}} & -f_{\mathrm{x}} - \frac{f_{\mathrm{x}}X^{\mathrm{c}2}}{Z^{\mathrm{c}2}} & \frac{f_{\mathrm{x}}Y^{\mathrm{c}}}{Z^{\mathrm{c}}} \\ 0 & -\frac{f_{\mathrm{y}}}{Z^{\mathrm{c}}} & \frac{f_{\mathrm{y}}Y^{\mathrm{c}}}{Z^{\mathrm{c}2}} & f_{\mathrm{y}} + \frac{f_{\mathrm{y}}Y^{\mathrm{c}2}}{Z^{\mathrm{c}2}} & -\frac{f_{\mathrm{y}}X^{\mathrm{c}}Y^{\mathrm{c}}}{Z^{\mathrm{c}2}} & -\frac{f_{\mathrm{y}}X^{\mathrm{c}}}{Z^{\mathrm{c}}} \end{bmatrix}. \tag{29}$$

To obtain $\boldsymbol{\xi}$, we substitute the Jacobian matrix (28) into the Gauss-Newton optimization method (see Algorithm 1).

Algorithm 1: Pose calibration.

Input: $\mathbf{p}_i^{\mathrm{w}}$, 3D coordinate of markers in Vicon frame;
$\mathbf{p}_i$, 2D coordinates of markers in the image;
Output: The 6D pose $\boldsymbol{\xi}$;
Initialize camera pose $\boldsymbol{\xi} = [0, 0, 0, 0, 0, 0]^\top$;
repeat
  Project $\mathbf{p}_i^{\mathrm{w}}$ to the image plane with $\mathbf{p}_i' = \frac{1}{s_i}\mathbf{K}\exp(\boldsymbol{\xi}^\wedge)\mathbf{p}_i^{\mathrm{w}}$;
  Compute reprojection error $\boldsymbol{e}_i = \mathbf{p}_i' - \mathbf{p}_i$ and Jacobian matrix $\boldsymbol{J}$ with (29);
  Compute $\boldsymbol{H} = \sum_{i=1}^{n} \boldsymbol{J}_i^\top \boldsymbol{J}$ and $\boldsymbol{b} = -\sum_{i=1}^{n} \boldsymbol{J}_i^\top \boldsymbol{e}_i$;
  Compute $\boldsymbol{\Delta}$ by solving equation $\boldsymbol{H}\boldsymbol{\Delta} = \boldsymbol{b}$;
  Update $\boldsymbol{\xi} \leftarrow \exp(\boldsymbol{\Delta})\boldsymbol{\xi}$;
until $||\boldsymbol{\Delta}||_2^2 < 10^{-6}$;

By substituting ϕ into the Rodrigues' rotation formula, the rotation matrix $\mathbf{R}_{\mathrm{w}}^{\mathrm{c}}$ can be obtained. Since $\boldsymbol{\rho}$ is equivalent to $\mathbf{t}_{\mathrm{w}}^{\mathrm{c}}$, then we can obtain the matrix $\mathbf{T}_{\mathrm{w}}^{\mathrm{c}} = [\mathbf{R}_{\mathrm{w}}^{\mathrm{c}}|\boldsymbol{\rho}] = [\mathbf{R}_{\mathrm{w}}^{\mathrm{c}}|\mathbf{t}_{\mathrm{w}}^{\mathrm{c}}]$.

REFERENCES

[1] Y. Zheng, Z. Chen, D. Lv, Z. Li, Z. Lan, and S. Zhao, "Air-to-air visual detection of micro-UAVs: An experimental evaluation of deep learning," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1020–1027, 2021.

[2] Y. Zheng, C. Zheng, X. Zhang, F. Chen, Z. Chen, and S. Zhao, "Detection, localization, and tracking of multiple MAVs with panoramic stereo camera networks," *IEEE Transactions on Automation Science and Engineering*, 2022.

[3] X. Shi, C. Yang, W. Xie, C. Liang, Z. Shi, and J. Chen, "Anti-drone system with multiple surveillance technologies: Architecture, implementation, and challenges," *IEEE Communications Magazine*, vol. 56, no. 4, pp. 68–74, 2018.

[4] M. A. Khan, H. Menouar, A. Eldeeb, A. Abu-Dayya, and F. D. Salim, "On the detection of unauthorized drones-techniques and future perspectives: A review," *IEEE Sensors Journal*, 2022.

[5] J. Xie, C. Gao, J. Wu, Z. Shi, and J. Chen, "Small low-contrast target detection: Data-driven spatiotemporal feature fusion and implementation," *IEEE Transactions on Cybernetics*, pp. 1–12, 2021.

[6] K. Guo, X. Li, and L. Xie, "Ultra-wideband and odometry-based cooperative relative localization with application to multi-UAV formation control," *IEEE Transactions on Cybernetics*, vol. 50, no. 6, pp. 2590–2603, 2020.

[7] A. Kushleyev, D. Mellinger, C. Powers, and V. Kumar, "Towards a swarm of agile micro quadrotors," *Autonomous Robots*, vol. 35, no. 4, pp. 287–300, 2013.

[8] G. Vásárhelyi, C. Virágh, G. Somorjai, T. Nepusz, A. E. Eiben, and T. Vicsek, "Optimized flocking of autonomous drones in confined environments," *Science Robotics*, vol. 3, no. 20, p. eaat3536, 2018.

[9] H. Liu, T. Ma, F. L. Lewis, and Y. Wan, "Robust formation trajectory tracking control for multiple quadrotors with communication delays," *IEEE Transactions on Control Systems Technology*, vol. 28, no. 6, pp. 2633–2640, 2020.

[10] F. Schilling, F. Schiano, and D. Floreano, "Vision-based drone flocking in outdoor environments," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 2954–2961, 2021.

[11] Y. Tang, Y. Hu, J. Cui, F. Liao, M. Lao, F. Lin, and R. S. Teo, "Vision-aided multi-UAV autonomous flocking in GPS-denied environment," *IEEE Transactions on Industrial Electronics*, vol. 66, no. 1, pp. 616–626, 2018.

[12] P. Zhang, G. Chen, Y. Li, and W. Dong, "Agile formation control of drone flocking enhanced with active vision-based relative localization," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6359–6366, 2022.

[13] M. R. Cohen and J. R. Forbes, "Navigation and control of unconventional VTOL UAVs in forward-flight with explicit wind velocity estimation," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1151–1158, 2020.

[14] M. Turpin, N. Michael, and V. Kumar, "Trajectory design and control for aggressive formation flight with quadrotors," *Autonomous Robots*, vol. 33, pp. 143–156, 2012.

[15] M. Shalaby, C. C. Cossette, J. R. Forbes, and J. Le Ny, "Relative position estimation in multi-agent systems using attitude-coupled range measurements," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4955–4961, 2021.

[16] W. Zhao, H. Liu, and F. L. Lewis, "Data-driven fault-tolerant control for attitude synchronization of nonlinear quadrotors," *IEEE Transactions on Automatic Control*, vol. 66, no. 11, pp. 5584–5591, 2021.

[17] Y. Zou and Z. Meng, "Coordinated trajectory tracking of multiple vertical take-off and landing UAVs," *Automatica*, vol. 99, pp. 33–40, 2019.

[18] I. Shugurov, F. Li, B. Busam, and S. Ilic, "OSOP: A multi-stage one shot object pose estimation framework," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 6835–6844.

[19] S. Zhang, W. Zhao, Z. Guan, X. Peng, and J. Peng, "Keypoint-graph-driven learning framework for object pose estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 1065–1073.

[20] T. Lee, B.-U. Lee, M. Kim, and I. S. Kweon, "Category-level metric scale object shape and pose estimation," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 8575–8582, 2021.

[21] Z. Fan, Y. Zhu, Y. He, Q. Sun, H. Liu, and J. He, "Deep learning on monocular object pose detection and tracking: A comprehensive overview," *ACM Computing Surveys*, vol. 55, no. 4, pp. 1–40, 2022.

[22] R. Jin, J. Jiang, Y. Qi, D. Lin, and T. Song, "Drone detection and pose estimation using relational graph networks," *Sensors*, vol. 19, no. 6, p. 1479, 2019.

[23] G. Albanis, N. Zioulis, A. Dimou, D. Zarpalas, and P. Daras, "Dronepose: Photorealistic UAV-assistant dataset synthesis for 3D pose estimation via a smooth silhouette loss," in *Proceedings of the European Conference on Computer Vision Workshop (ECCVW)*, 2020, pp. 663–681.

[24] M. Pavliv, F. Schiano, C. Reardon, D. Floreano, and G. Loianno, "Tracking and relative localization of drone swarms with a vision-based headset," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1455–1462, 2021.

[25] L. Vincent, M.-N. Francesc, and F. Pascal, "EPnP: An accurate O(n) solution to the PnP problem," *International Journal of Computer Vision*, 2009.

[26] B. Chen, T.-J. Chin, and M. Klimavicius, "Occlusion-robust object pose estimation with holistic representation," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2022, pp. 2929–2939.

[27] B. Tekin, S. N. Sinha, and P. Fua, "Real-time seamless single shot 6D object pose prediction," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 292–301.

[28] Y. Bukschat and M. Vetter, "EfficientPose: An efficient, accurate and scalable end-to-end 6D multi object pose estimation approach," *arXiv preprint arXiv:2011.04307*, 2020.

[29] Y. Di, F. Manhardt, G. Wang, X. Ji, N. Navab, and F. Tombari, "SO-Pose: Exploiting self-occlusion for direct 6D pose estimation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (CVPR)*, 2021, pp. 12 396–12 405.

[30] W. Kehl, F. Manhardt, F. Tombari, S. Ilic, and N. Navab, "SSD-6D: Making RGB-based 3D detection and 6D pose estimation great again," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 1521–1529.

[31] P. Liu, Q. Zhang, J. Zhang, F. Wang, and J. Cheng, "MFPN-6D: Real-time one-stage pose estimation of objects on RGB images," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 12 939–12 945.

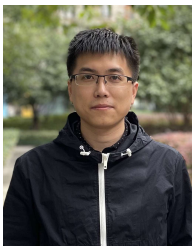
[32] Y. Hu, S. Speierer, W. Jakob, P. Fua, and M. Salzmann, "Wide-depth-range 6D object pose estimation in space," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 15 870–15 879.

[33] C.-P. Lu, G. D. Hager, and E. Mjolsness, "Fast and globally convergent pose estimation from video images," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 6, pp. 610–622, 2000.

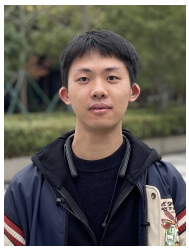
[34] G. Pavlakos, X. Zhou, A. Chan, K. G. Derpanis, and K. Daniilidis, "6-DoF object pose from semantic keypoints," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 2011–2018.

[35] S. Ben-David and R. Urner, "On the hardness of domain adaptation and the utility of unlabeled target samples," in *Proceedings of the*

International Conference on Algorithmic Learning Theory (ALT), 2012, pp. 139–153.
[36] H. Liu, J. Wang, and M. Long, "Cycle self-training for domain adaptation," *Proceedings of the Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 22 968–22 981, 2021.
[37] Y. You, C. A. Diaz-Ruiz, Y. Wang, W.-L. Chao, B. Hariharan, M. Campbell, and K. Q. Weinbergert, "Exploiting playbacks in unsupervised domain adaptation for 3D object detection in self-driving cars," in *Proceedings of the International Conference on Robotics and Automation (ICRA)*, 2022, pp. 5070–5077.
[38] R. A. Marsden, A. Bartler, M. Döbler, and B. Yang, "Contrastive learning and self-training for unsupervised domain adaptation in semantic segmentation," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, 2022, pp. 1–8.
[39] X. Liu, C. Yoo, F. Xing, H. Oh, G. El Fakhri, J.-W. Kang, J. Woo *et al.*, "Deep unsupervised domain adaptation: a review of recent advances and perspectives," *APSIPA Transactions on Signal and Information Processing*, vol. 11, no. 1, 2022.
[40] Y. Zou, Z. Yu, X. Liu, B. Kumar, and J. Wang, "Confidence regularized self-training," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 5982–5991.
[41] J. Yang, S. Shi, Z. Wang, H. Li, and X. Qi, "ST3D: Self-training for unsupervised domain adaptation on 3D object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2021, pp. 10 368–10 378.
[42] G. Ding, M. Zhang, E. Li, and Q. Hao, "JST: Joint self-training for unsupervised domain adaptation on 2D&3D object detection," in *Proceedings of the International Conference on Robotics and Automation (ICRA)*, 2022, pp. 477–483.
[43] Y. Zhang, M. Li, Y. Xie, C. Li, C. Wang, Z. Zhang, and Y. Qu, "Self-supervised exclusive learning for 3D segmentation with cross-modal unsupervised domain adaptation," in *Proceedings of the ACM International Conference on Multimedia (ACM MM)*, 2022, pp. 3338–3346.
[44] C. Li, L. Li, H. Jiang, K. Weng, Y. Geng, L. Li, Z. Ke, Q. Li, M. Cheng, W. Nie *et al.*, "YOLOv6: A single-stage object detection framework for industrial applications," *arXiv preprint arXiv:2209.02976*, 2022.
[45] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, "RepVGG: Making VGG-style convnets great again," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 13 733–13 742.
[46] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 8759–8768.
[47] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.
[48] E. Brachmann, A. Krull, F. Michel, S. Gumhold, J. Shotton, and C. Rother, "Learning 6D object pose estimation using 3D object coordinates," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014, pp. 536–551.
[49] S. Hinterstoisser, V. Lepetit, S. Ilic, S. Holzer, G. Bradski, K. Konolige, and N. Navab, "Model based training, detection and pose estimation of texture-less 3D objects in heavily cluttered scenes," in *Proceedings of the Asian Conference on Computer Vision (ACCV)*, 2012, pp. 548–562.
[50] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.
[51] S. Peng, Y. Liu, Q. Huang, X. Zhou, and H. Bao, "PVNet: Pixel-wise voting network for 6DoF pose estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4561–4570.
[52] K. Park, T. Patten, and M. Vincze, "Pix2pose: Pixel-wise coordinate regression of objects for 6D pose estimation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 7668–7677.
[53] Y. Hu, P. Fua, W. Wang, and M. Salzmann, "Single-stage 6D object pose estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 2930–2939.
[54] J. Wang, T. Zhang, N. Sebe, H. T. Shen *et al.*, "A survey on learning to hash," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 4, pp. 769–790, 2017.



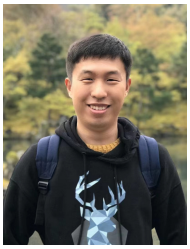
Ye Zheng received the B.S. degree from Guilin University of Technology, Guilin, China, in 2016, and the M.S. degree in the Department of Electromechanical and Automation from Harbin Institute of Technology, Shenzhen, China, in 2019. He is currently pursing the Ph.D. degree in Computer Science in the School of Engineering, Westlake University, Hangzhou, China, and joint-cultivated by Zhejiang University, Hangzhou, China. His research interests lie in computer vision and robotics.



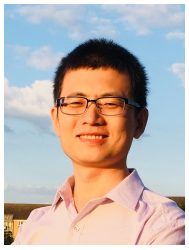
Canlun Zheng received the B.S. degree in Mechanical Engineering from Shandong University, Shandong, China, in 2017, and the M.S. degree in the Department of Astronautics Engineering from Beihang University, Beijing, China, in 2020. He is currently pursing the Ph.D. degree in Computer Science in the School of Engineering, Westlake University, Hangzhou, China, and joint-cultivated by Zhejiang University, Hangzhou, China. His research interests include UAV control, motion tracking, and robotics.



Jiahao Shen received the B.S. degree from China Jiliang University, Hangzhou, China, in 2019, and the M.S. degree in the Department of Automation Engineering from Nanjing University of Aeronautics and Astronautics, Nanjing, China. Since 2022, he has been working as a Research Assistant with the Intelligent Unmanned Systems Laboratory, Westlake University. His research includes system modeling and flight control design.



Peidong Liu received the B.E. and M.E. degrees from the National University of Singapore, in 2012 and 2015, respectively. He then joined ETH Zurich as a postgraduate student and received his PhD degree in Computer Science in 2021. He is currently an Assistant Professor with the School of Engineering, Westlake University, Hangzhou, China. His research interests lie in 3D computer vision and robotics.



Shiyu Zhao received the B.E. and M.E. degrees from Beijing University of Aeronautics and Astronautics in 2006 and 2009, respectively, and the Ph.D. degree from the National University of Singapore in 2014. He is currently an Associate Professor in the School of Engineering and the Director of the Intelligent Unmanned Systems Laboratory at Westlake University, Hangzhou, China. Before he joined Westlake University, he was a Lecturer in the Department of Automatic Control and Systems Engineering at the University of Sheffield, UK. His research interests lie in sensing, estimation, and control of multi-robot systems.